🧾 Flutter Form Validation (Child → Parent)

🎯 Goal

Understand how to: - Pass validator from **child widget** - Control validation from **parent widget** - Use `GlobalKey<FormState>` properly

---

🧠 Core Concept

In Flutter:

```
Parent controls the Form
Child defines the Fields
```

So: - Parent → holds `FormKey` - Child → uses that key inside `Form`

🧩 Step 1: Create Form Key in Parent

```dart
final _formKey = GlobalKey<FormState>();
```

🧩 Step 2: Pass FormKey to Child

```dart
Step1Body(
  formKey: _formKey,
)
```

🧩 Step 3: Use Form in Child

```dart
class Step1Body extends StatelessWidget {
  final GlobalKey<FormState> formKey;

  const Step1Body({required this.formKey});
```

```
  @override
  Widget build(BuildContext context) {
    return Form(
      key: formKey,
      child: Column(
        children: [
          TextFormField(
            validator: InputValidator.validateField,
          ),
        ],
      ),
    );
  }
}
```

🧩 Step 4: Validator Function

```
class InputValidator {
  static String? validateField(String? value) {
    if (value == null || value.trim().isEmpty) {
      return "This field is required";
    }
    return null;
  }
}
```

🧩 Step 5: Trigger Validation from Parent

```
if (_formKey.currentState!.validate()) {
  // valid → go next
} else {
  // invalid → show errors
}
```

🔄 Your Flow (Real Example)

Parent (BookTransfer)

```
void _onContinue() {
  if (currentStep == 1) {
    if (_step1FormKey.currentState!.validate()) {
      setState(() => currentStep++);
    }
  }
}
```

Child (Step1Body)

```
LocationInputField(
  validator: InputValidator.validateField,
)
```

⚙️ How Data Flows

```
User Input
   ↓
TextFormField
   ↓
Validator (Child)
   ↓
FormKey.validate() (Parent)
   ↓
Result (true / false)
```

❗ Common Mistakes

❌ 1. Creating FormKey in Child

Wrong ❌

```
final _formKey = GlobalKey<FormState>();
```

✔️ Correct → Always in Parent

❌ 2. Not Passing FormKey

Form won't validate if key not shared

❌ 3. Using TextField instead of TextFormField

Validator works only with:

```
TextFormField ✅
TextField ❌
```

🚀 Advanced (Optional)

Custom Validator per Field

```
validator: (value) {
  if (value!.length < 3) return "Too short";
  return null;
}
```

Multiple Forms (Like Your Case)

```
final _step1FormKey = GlobalKey<FormState>();
final _step2FormKey = GlobalKey<FormState>();
```

Each step has separate validation ✔️

✅ Summary

- Parent holds `FormKey`
- Child uses `Form`
- Validators stay in child
- Parent calls `.validate()`
- Navigation depends on validation

💡 Pro Tip

You can also use:

- `autovalidateMode: AutovalidateMode.onUserInteraction`

```
Form(
  key: formKey,
  autovalidateMode: AutovalidateMode.onUserInteraction,
)
```

🏁 Final Result

✔️ Clean architecture
✔️ Reusable widgets
✔️ Centralized validation control
✔️ Scalable multi-step forms

If you want next level, I can show:

🔥 FormController pattern
🔥 Riverpod / Bloc form validation
🔥 Real-time validation UX
🔥 Production booking flow structure